

A project

PHARMACY INVENTORY MANAGEMENT SYSTEM



CSE 100: Software Development Project I

SUBMITTED BY

Name	ID	Intake
MAKHZUM-BIN-HARUN	20254103279	56
MD. RAFIUL ISLAM	20254103269	56
ESTIAK AHMED TURNAB	20254103272	56

SUPERVISED BY:

Mastura Sadaf

Lecturer

Department of CSE

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
BANGLADESH UNIVERSITY OF BUSINESS AND TECHNOLOGY (BUBT)**

August 20, 2026

ABSTRACT

Most small and medium-sized pharmacies in Bangladesh still manage their medicine stock and sales records manually, using paper registers or simple spreadsheet files. This creates recurring problems such as inaccurate stock counts, delayed awareness of low-stock medicines, error-prone manual sales records, and no control over who is allowed to change stock or pricing data. This project presents a console-based Pharmacy Inventory Management System, developed in C using structured programming and binary file handling, to address these problems for a small pharmacy without requiring internet access or paid software.

The system supports two user roles, Admin and Staff, with role-based access control: both roles can log in, view and search medicines, sell medicine, and generate reports, while adding, updating, or deleting medicine records requires either an Admin login or Staff entering the admin password. The system automatically tracks stock levels, blocks sales that exceed available stock, warns when a medicine's quantity falls to 10 units or below, and records every sale with a unique, sequentially generated Transaction ID. Three reports, Inventory, Sales History, and Low Stock, can be generated from within the system. Beyond the original proposal, the final system also allows the admin to register additional Staff or Admin accounts, rather than being limited to a single fixed account of each type.

The system was built using the Incremental Development Model, across four increments completed by a three-member team, and was tested on Windows using GCC. This report describes the background, design, implementation, and testing of the completed system, along with its known limitations and possible directions for future work.

LIST OF FIGURES

Sl No.	Figure Name	Page
01	<i>Figure 3.1: Use Case Diagram</i>	08
02	<i>Figure 3.2: Context Level Diagram (DFD Level 0)</i>	08
03	<i>Figure 3.3: Data Flow Diagram (Level 1)</i>	09
04	<i>Figure 3.4: File / Struct Schema for Users.dat, Medicines.dat, and Sales.dat</i>	10
05	<i>Figure 3.5: Flowchart for the Sell Medicine Process</i>	11
06	<i>Figure 5.1: First Prompt to Register an Admin if not added</i>	16
07	<i>Figure 5.2: Welcome Screen</i>	17
08	<i>Figure 5.3: Admin Login</i>	17
09	<i>Figure 5.4: Admin Dashboard</i>	17
10	<i>Figure 5.5: Add Medicine (Admin)</i>	17
11	<i>Figure 5.6: Sell Medicine</i>	18
12	<i>Figure 5.7: Inventory Report</i>	18
13	<i>Figure 5.8: Sales Report</i>	19
14	<i>Figure 5.9: Register New User</i>	19
15	<i>Figure 5.10: Staff Dashboard</i>	19
16	<i>Figure 5.11: Add Medicine (Staff, Admin password prompt)</i>	20
17	<i>Figure 5.12: Sell Medicine</i>	20

TABLE OF CONTENTS

ABSTRACT.....	ii
LIST OF FIGURES	iii
TABLE OF CONTENTS	iv
Chapter 1 INTRODUCTION	1-3
1. 1. Problem Specification/Statement.....	1
1.2. Objectives	1
1.3. Scope.....	2
1.4. Organization Of Project Report	2
Chapter 2 BACKGROUND	4-5
2.1. Existing System Analysis.....	4
2.2. Supporting Literatures	4
Chapter 3 SYSTEM ANALYSIS & DESIGN	6-12
3.1. Technology & Tools	6
Programming Language.....	6
Development Environment	6
Data Storage.....	6
Hardware Requirements.....	6
3.2. Model & Diagram	6
3.2.1 Model (Incremental Development Model)	6
3.2.2 Use Case Diagram.....	7
3.2.3 Context Level Diagram.....	8
3.2.4 Data Flow Diagram.....	9
3.2.5 Database Schema	9
3.2.6 Algorithms/Flowchart	11
Chapter 4 IMPLEMENTATION	13-15
4.1 Interface Design/Front-End	13
4.2 Back-End.....	13
4.3 Modules.....	14
Authentication (auth.h / auth.c)	14
Medicine Management (medicine.h / medicine.c).....	14
Inventory Management (inventory.h / inventory.c)	14
Sales Management (sales.h / sales.c)	14

Reports (reports.h / reports.c)	15
Utilities (utils.h / utils.c)	15
Chapter 5 USER MANUAL	16-20
5.1 System Requirement	16
5.1.1 Hardware Requirement	16
5.1.2 Software Requirement	16
5.2 User Interfaces	16
5.2.1 Panel A — Admin Interface	16
5.2.2 Panel B — Staff Interface	19
5.2.3 Login Credentials.....	20
Chapter 6 CONCLUSION	21-22
6.1 Conclusion	21
6.2 Limitation.....	21
6.3 Future Works.....	21
REFERENCES.....	22
APPENDIX A.....	22
Appendix A	22
A.1 Project Summary.....	23
A.2 Technology Stack	23
A.3 Repository Structure	24
A.4 How to Build and Run	24
A.5 License	24

Chapter 1

INTRODUCTION

1. 1. Problem Specification/Statement

Most small and medium-sized pharmacies in Bangladesh still manage their medicine stock and sales records manually. A pharmacist or sales staff typically keeps track of available medicines using paper registers or simple spreadsheet files. This approach causes several recurring problems: stock records are hard to maintain manually, so when a medicine is sold, the stock count is often not updated right away, leading to confusion about what is actually available. There is no easy way to know which medicines are running low until they are completely out of stock, by which point customers may have already been turned away. Sales transactions recorded by hand take time, introduce errors, and make it difficult to find a particular past sale. A manual system also has no access control, so any employee can change stock or pricing data with no record of who made the change. Generating any kind of report, such as which medicines are near running out, requires significant manual effort.

This project was undertaken to solve these problems by building a simple, offline, console-based system that automates the core inventory and sales tasks of a pharmacy, while keeping a controlled access model so that sensitive operations require proper authorization.

1.2. Objectives

The specific objectives of this project were:

- Build a role-based login system that separates Admin and Staff access, so that each user only gets the features they are authorized to use.
- Allow the Admin to add, update, search, and delete medicine records, while Staff members can perform these sensitive operations only after entering the admin password.
- Track stock quantities for each medicine and automatically reduce stock whenever a sale is made.
- Prevent sales from going through if a medicine is out of stock, so the system always reflects accurate availability.
- Alert the user whenever a medicine's stock falls to 10 units or below, giving enough time to reorder before it runs out.
- Record every sale with a unique Transaction ID, so individual transactions can be traced if needed.
- Allow both Admin and Staff to generate inventory, sales history, and low stock reports from within the system.
- Store all data in binary .dat files using structured file handling in C, without requiring any external database software.

How far each of these objectives was met is discussed in Chapter 6.

The Pharmacy Inventory Management System is a console-based application developed in C. The aim is to build a working system that a small pharmacy can use to manage their daily medicine inventory and sales operations without needing internet access or any paid software.

The specific objectives of this project are:

- Build a role-based login system that separates Admin and Staff access, so that each user only gets the features they are authorized to use.
- Allow the Admin to add, update, search, and delete medicine records, while Staff members can perform these sensitive operations only after entering the admin password.
- Track stock quantities for each medicine and automatically reduce stock whenever a sale is made.
- Prevent sales from going through if a medicine is out of stock, so the system always reflects accurate availability.
- Alert the user whenever a medicine's stock falls below 10 units, giving enough time to reorder before it runs out.
- Record every sale with a unique Transaction ID, so individual transactions can be traced if needed.
- Allow both Admin and Staff to generate inventory, sales history, and low stock reports from within the system.
- Store all data in binary .dat files using structured file handling in C, without requiring any external database software.

1.3. Scope

The project began with identifying the problems faced by small pharmacies that rely on manual record-keeping, followed by writing and submitting a project proposal that defined the feature scope, technology stack, and development model. Once the proposal was approved by our supervisor, development proceeded using the Incremental Development Model, across four increments: Login system and file structure setup; Medicine management; Inventory tracking and Sales; and finally Reports with integration and testing.

Each increment was implemented, compiled, and tested on its own before the next one began. Version control was maintained through Git and GitHub throughout, with feature work done on a development branch and merged into the main branch after review. Once all four increments were complete and tested together, the team prepared a presentation and was selected to represent the class at the Software Development Idea Pitching & Project Showcasing event at BUBT. This report was written after the project and its presentation were completed.

1.4. Organization Of Project Report

The rest of this report is organized as follows:

- Chapter 2 (Background) reviews existing pharmacy-related software and the supporting literature and tools used in this project.
- Chapter 3 (System Analysis & Design) describes the technology stack, development model, and the system's design through use case, context level, and data flow diagrams, along with the file schema and a sample algorithm.
- Chapter 4 (Implementation) describes how the interface, back-end, and individual modules of the system were actually built.

- Chapter 5 (User Manual) describes the hardware and software requirements to run the system and walks through its interfaces for both the Admin and Staff roles.
- Chapter 6 (Conclusion) summarizes the project, its limitations, and possible future improvements.

Chapter 2

BACKGROUND

2.1. Existing System Analysis

Before designing this system, we looked at a few existing pharmacy-related software solutions that are commonly used by small pharmacies in Bangladesh, to understand what features they provide and where they fall short. The table below summarizes the two main types of systems we studied.

System Type	Category	Key Limitations Relevant to Our Project
Local Pharmacy POS Tools	Basic billing software (Excel / Windows Forms)	Handles billing only. No inventory integration, no role-based access control, no low stock alerts. Stock must be tracked separately. Files are prone to accidental corruption or deletion.
Medi-Soft	GUI-based pharmacy management software	Requires a paid license, making it unaffordable for small pharmacies. Heavyweight GUI with specific hardware requirements. Contains many features irrelevant to typical Bangladeshi pharmacy operations.

Neither of the systems above provides a combined solution for offline inventory management, role-based access control, low stock alerts, and sales tracking within a single program. Our system addresses these gaps by combining all of these features into one lightweight, offline, console-based application. It does not require any license fee, does not need a network connection, and runs on any standard Windows PC that has GCC installed, which makes it realistic for a small pharmacy to actually use.

2.2. Supporting Literatures

The implementation of this project relies on standard C programming concepts covered in our coursework, particularly structured programming and low-level file handling. Kernighan and Ritchie's *The C Programming Language* [1] was used as the primary reference for correct usage of file I/O functions such as `fopen()`, `fread()`, `fwrite()`, and `fseek()`, which form the basis of how our system stores and retrieves data from `.dat` files.

For the choice of development process, we referred to Pressman's *Software Engineering: A Practitioner's Approach* [4], which describes the Incremental Development Model in detail, and to Royce's original paper on managing large software systems [3], which is commonly cited as background for staged software development approaches. These sources supported our decision to build and test the system in four separate increments rather than attempting to build the whole system at once.

During development and testing, MedEx Bangladesh [2], an online medicine information database for the Bangladeshi market, was used as a reference to enter realistic medicine names, generic names, and

categories (for example, Finix 20mg with the generic name Rabeprazole Sodium) instead of placeholder test data, so that the system could be demonstrated with data that resembles what a real pharmacy would actually store.

Official documentation for GCC [5] and Visual Studio Code [6] was used throughout development for compiler flags, debugging, and editor configuration.

Chapter 3

SYSTEM ANALYSIS & DESIGN

3.1. Technology & Tools

The following tools and technologies were used to build and test the final system:

Programming Language

- C (C11 standard)
- Standard C libraries used: `stdio.h`, `stdlib.h`, `string.h`, `time.h`

Development Environment

- Compiler: GCC via MinGW-w64 (Windows)
- Editor: Visual Studio Code with the C/C++ extension
- Operating System: Windows 10 or later (primary development and testing environment)
- Version Control: Git and GitHub (repository: github.com/makhzumbinharun/pharmacy-inventory-system)

Data Storage

- Binary `.dat` files, accessed using C's structured file handling functions (`fread`, `fwrite`, `fseek`)
- Separate `.dat` files for user accounts, medicine records, and sales transactions

Hardware Requirements

- Any PC or laptop with at least 2 GB RAM
- At least 500 MB free disk space for the compiler, editor, and project files
- Standard keyboard and monitor

The system does not require an internet connection at any point, since all data is stored and processed locally on the machine it runs on.

3.2. Model & Diagram

3.2.1 Model (Incremental Development Model)

We used the Incremental Development Model to build this project. In this model, the system is built in separate stages, where each stage adds a new working module to the existing system, and each module is designed, coded, and tested before the next one begins.

We chose this model for a few practical reasons:

- The project naturally splits into five modules — Login, Medicine Management, Inventory, Sales, and Reports — each of which could be built and tested on its own.
- It meant that even under time pressure near the semester deadline, we would still have a partially working, demonstrable system rather than nothing at all.

- Bugs were easier to isolate and fix within a small, recently-built module than they would have been in a system built and tested all at once.
- It allowed the three of us to work on different modules at the same time without constantly blocking each other.

The project was completed in four increments:

- Increment 1: Login system and file structure setup
- Increment 2: Medicine management module (add, view, search, update, delete)
- Increment 3: Inventory tracking and Sales module
- Increment 4: Reporting module, followed by final integration and testing

This approach worked well for our team in practice. Each increment was tested on its own before we moved to the next one, which meant that by the time we reached integration and final testing, most of the individual modules were already stable.

3.2.2 Use Case Diagram

The system has two actors: Admin and Staff. Both roles can log in, view and search medicines, sell medicine, and generate reports. Admin can additionally add, update, and delete medicine records, and register new user accounts. Staff can also perform add, update, and delete operations, but only after the Verify Admin Password use case succeeds, which is why these three use cases are shown with an <<include>> relationship to Verify Admin Password in the diagram below.

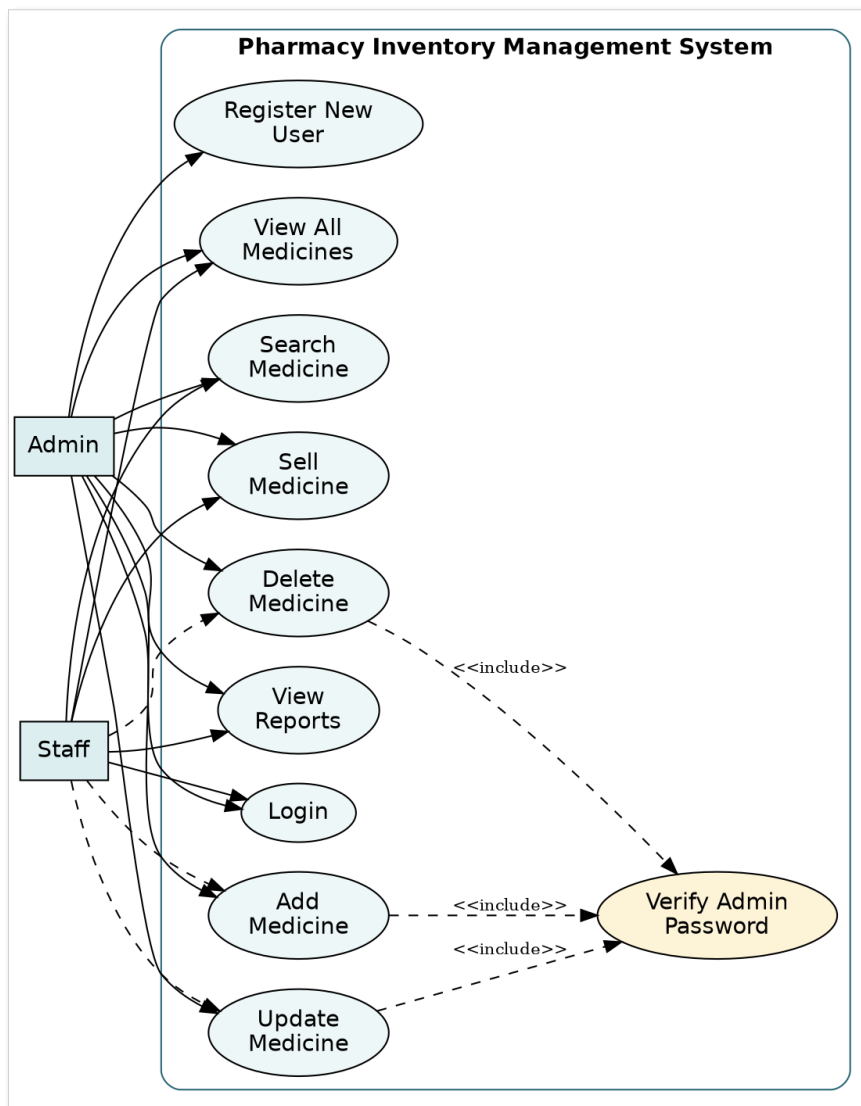


Figure 3.1: Use Case Diagram

3.2.3 Context Level Diagram

The context level diagram (DFD Level 0) shows the system as a single process and its interaction with the two external entities, Admin and Staff. Both entities send login credentials, medicine data, sale data, and (for Admin) user registration data into the system, and receive reports, confirmation messages, and stock alerts back.

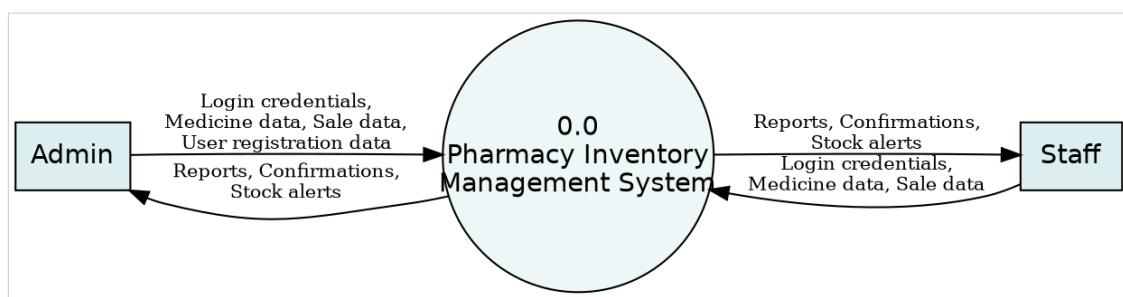


Figure 3.2: Context Level Diagram (DFD Level 0)

3.2.4 Data Flow Diagram

The Level 1 Data Flow Diagram breaks the system down into its five main processes: Authentication, Medicine Management, Inventory Management, Sales Management, and Reports Generation. These processes read from and write to three data stores, which correspond directly to the three .dat files used by the system: Users.dat, Medicines.dat, and Sales.dat.

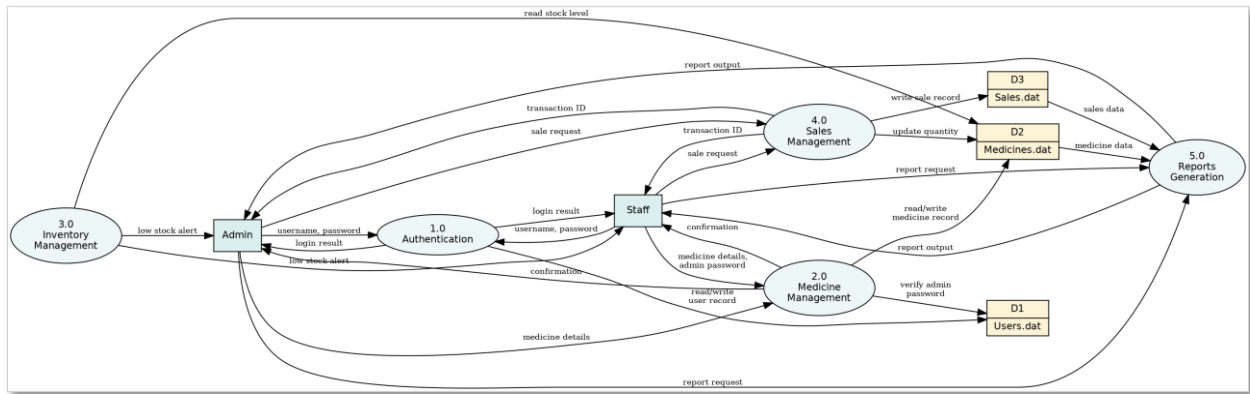


Figure 3.3: Data Flow Diagram (Level 1)

3.2.5 Database Schema

Since the system uses binary file handling instead of a relational database, there is no traditional database schema with tables and foreign keys. Instead, each .dat file stores a sequence of fixed-size records, where each record's layout is defined by a C struct. The figure below shows the three structs used in the final implementation, along with the size of each field in bytes.

Users.dat (struct User)		
Field	Type	Size
username	char[]	21 bytes
password	char[]	21 bytes
role	char	1 byte ('A'/'S')

Medicines.dat (struct Medicine)		
Field	Type	Size
id	int	4 bytes
name	char[]	51 bytes
genericName	char[]	51 bytes
category	char[]	31 bytes
price	float	4 bytes
quantity	int	4 bytes

Sales.dat (struct Sale)		
Field	Type	Size
transactionId	char[]	8 bytes
medicineId	int	4 bytes
medicineName	char[]	51 bytes
quantitySold	int	4 bytes
unitPrice	float	4 bytes
totalAmount	float	4 bytes
date	char[]	12 bytes
username	char[]	21 bytes

Figure 3.4: File / Struct Schema for Users.dat, Medicines.dat, and Sales.dat

Each record is written and read as a whole using `fwrite()` and `fread()`, which is why every field has a fixed size — C needs to know the exact size of a struct in bytes to correctly read and write it as a block of binary data.

3.2.6 Algorithms/Flowchart

The flowchart below illustrates the algorithm used in the Sell Medicine process, which is one of the more involved processes in the system since it touches both the Medicines.dat and Sales.dat files and includes stock validation.

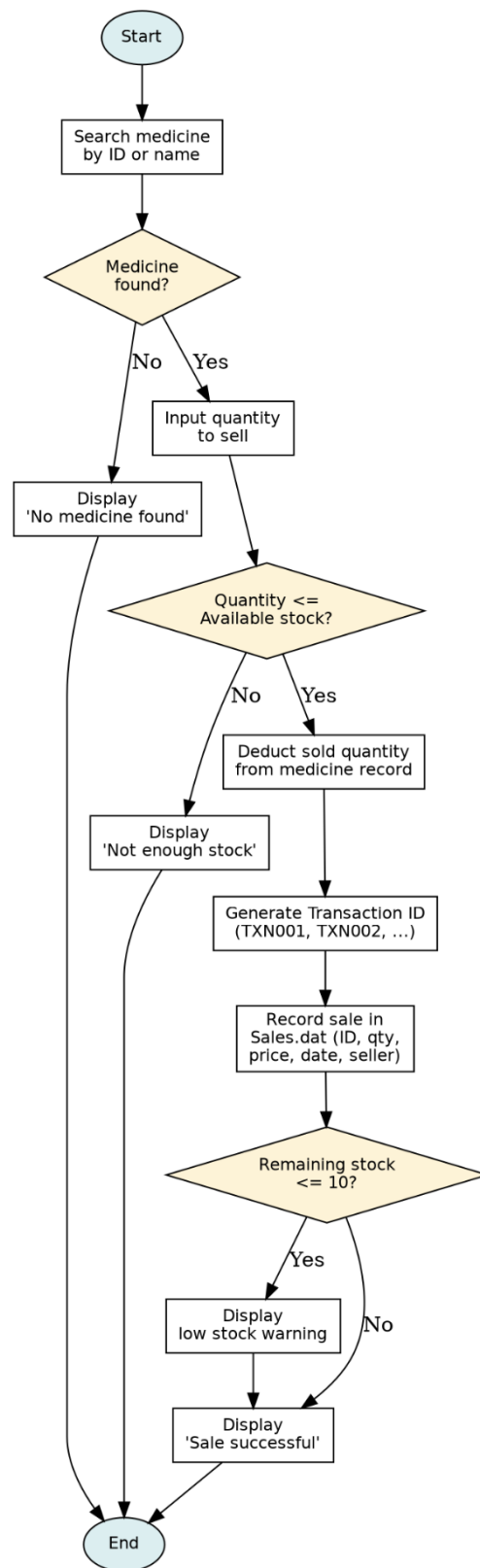


Figure 3.5: Flowchart for the Sell Medicine Process

The steps shown in the flowchart can also be written out as follows:

1. Search for the medicine by ID or name.
2. If no matching medicine is found, display an error message and stop.
3. Otherwise, ask the user for the quantity to sell.
4. If the requested quantity is more than the available stock, display 'Not enough stock' and stop.
5. Otherwise, deduct the sold quantity from the medicine's stock and save the updated record.
6. Generate the next sequential Transaction ID (e.g. TXN001, TXN002, ...).
7. Record the sale in Sales.dat, including the transaction ID, medicine details, quantity, unit price, total amount, date, and the username of the person who made the sale.
8. If the remaining stock is now 10 units or below, display a low stock warning.
9. Display a sale confirmation message and end.

Chapter 4

IMPLEMENTATION

4.1 Interface Design/Front-End

Since this is a console-based application, the front-end consists of text menus rather than graphical windows. The interface follows a consistent structure throughout the system: every screen prints a bordered header, a numbered list of options, and a prompt for input, which the user answers by typing a number.

After login, the user is taken to a Dashboard screen that lists Medicine Management, Sales Management, and Reports, with a Register New User option shown only when the logged-in user is an Admin. This means the Staff and Admin see slightly different menus without needing two separate programs — the menu is built dynamically based on the role stored at login.

A small utilities module was added purely for interface polish. It clears the screen between menu transitions so old output does not linger on screen, pauses with a "Press Enter to continue..." prompt so the user has time to read a result before the screen clears, and centers the welcome banner text based on the terminal's actual width at runtime, rather than assuming a fixed screen size.

4.2 Back-End

The back-end of the system is built entirely around C's structured file handling functions: `fopen()`, `fread()`, `fwrite()`, and `fseek()`. Each data type used by the system, User, Medicine, and Sale, is defined as a C struct with fixed-size fields, and each record is written to or read from its .dat file as a single fixed-size block.

A few back-end techniques are used repeatedly across modules:

- Adding a record (a new medicine, a new sale, a new user) opens its file in append mode ("ab"), which also creates the file automatically the first time it is used.
- Updating a record, such as changing a medicine's price or a medicine's ID, opens the file in read-and-write mode ("r+b"), reads through the file until the matching record is found, then uses `fseek()` to move the file position back to the start of that record before writing the updated data in place.
- Deleting a record cannot be done in place in a binary file, since removing bytes from the middle would corrupt the rest of the file. Instead, `deleteMedicine()` reads every record into a temporary file except the one being deleted, then replaces the original file with the temporary one.
- Duplicate-ID checks are performed before adding or updating a medicine, and a duplicate-username check is performed before registering a new user, to prevent two records from sharing the same identifier.
- Transaction IDs are generated automatically by counting how many sale records already exist in Sales.dat and formatting the next one as TXN001, TXN002, and so on.
- The date of each sale is captured automatically at the time of sale using C's `time.h` library.

4.3 Modules

The system is organized into the following modules, each with its own header (.h) and source (.c) file:

Authentication (auth.h / auth.c)

Handles login for both Admin and Staff against Users.dat. On the very first run, instead of shipping the program with a fixed default password, the system interactively asks whoever runs it for the first time to choose the initial Admin username and password. Two session variables, the current user's role and username, are set once at login and used by other modules for permission checks and for recording who performed an action.

As an enhancement beyond the original proposal, this module also includes registerUser(), which lets the logged-in Admin create additional Admin or Staff accounts at any time. The original proposal listed multiple user accounts as a possible future improvement and noted the system would support only a single fixed Admin and Staff account; in the final implementation, this was built and is a working part of the system rather than a planned addition.

Medicine Management (medicine.h / medicine.c)

Provides Add, View, Search, Update, and Delete operations for Medicines.dat. Each medicine record stores an ID, name, generic name, category, price, and quantity. The generic name field was added during implementation, beyond the field list in the original proposal, so that a medicine's brand name (for example, Finix 20mg) and its generic name (Rabeprazole Sodium) could both be recorded, which is closer to how medicines are actually labelled and searched for in practice.

Search, Update, and Delete all support finding a medicine either by its exact ID or by typing part of its name, rather than requiring the user to remember the exact ID every time. When a name search matches more than one medicine, the system lists the matches and asks the user to confirm the exact ID before proceeding.

Inventory Management (inventory.h / inventory.c)

Reuses the Medicine struct and Medicines.dat directly rather than keeping a separate inventory data store. It provides a low-stock check, checkLowStock(), and a low-stock listing, viewLowStockMedicines(), based on a fixed threshold of 10 units. This same check is reused by Sales Management to warn the user immediately after a sale reduces stock to that level.

Sales Management (sales.h / sales.c)

Handles selling a medicine: finding the medicine, validating that enough stock is available, deducting the sold quantity from Medicines.dat, generating a Transaction ID, and recording the sale in Sales.dat with the medicine's ID and name, quantity sold, unit price, total amount, date, and the username of whoever made the sale. If the requested quantity exceeds the available stock, the sale is blocked and no files are modified.

Reports (reports.h / reports.c)

Provides three reports: an Inventory Report and a Low Stock Report, both of which call the same functions already used in Medicine and Inventory Management rather than duplicating that logic, and a Sales Report, which reads through Sales.dat and prints every past transaction along with a running total of transactions and revenue.

Utilities (utils.h / utils.c)

A small module containing screen-clearing, pause, and text-centering functions used across the other modules, described in Section 4.1.

Chapter 5

USER MANUAL

5.1 System Requirement

5.1.1 Hardware Requirement

- Any PC or laptop with at least 2 GB RAM
- At least 500 MB free disk space for the compiler, editor, and project files
- Standard keyboard and monitor

5.1.2 Software Requirement

- Operating System: Windows 10 or later (primary environment; also tested on Linux with GCC, though Linux compatibility is not a strict requirement)
- Compiler: GCC via MinGW-w64 or MSYS2 (Windows) / GCC via package manager (Linux)
- Editor: Visual Studio Code with the C/C++ extension
- Git and GitHub, if the source code is obtained from the project repository

5.2 User Interfaces

The screenshots in this section are placeholders. They will be replaced with actual screenshots of the running program before final submission.

5.2.1 Panel A — Admin Interface

The admin sees the full set of options, including the ability to register new users.

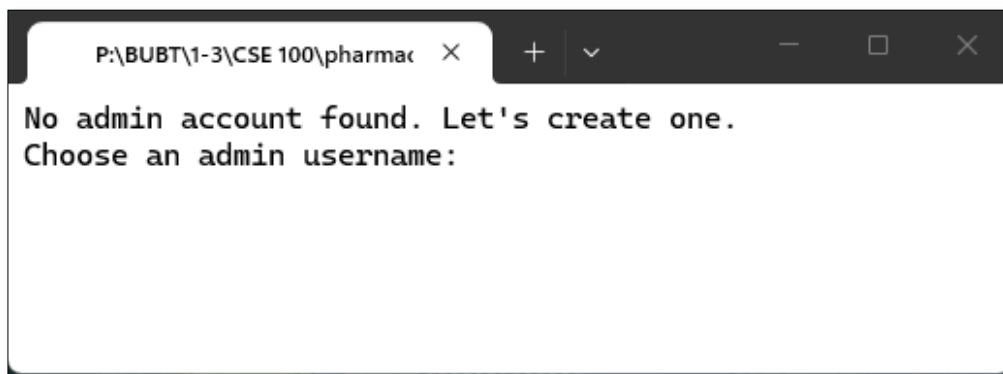


Figure 5.1: First Prompt to Register an Admin if not added

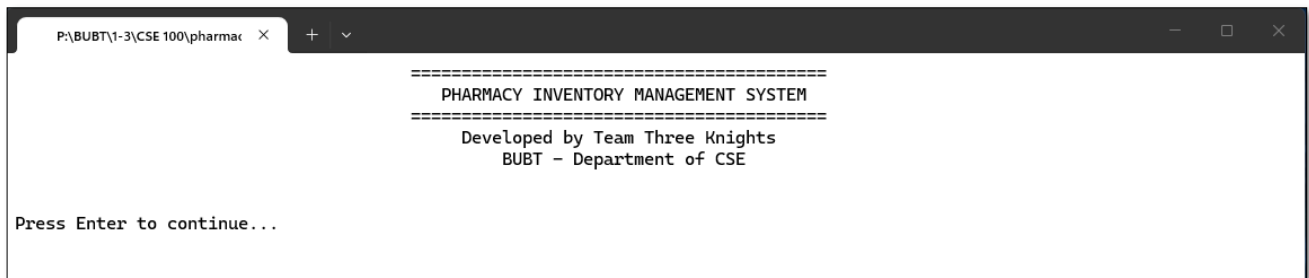


Figure 5.2: Welcome Screen

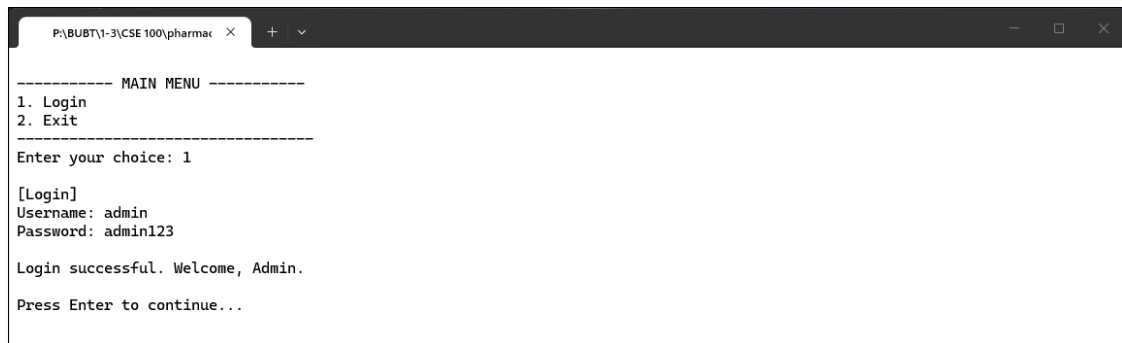


Figure 5.3: Admin Login



Figure 5.4: Admin Dashboard

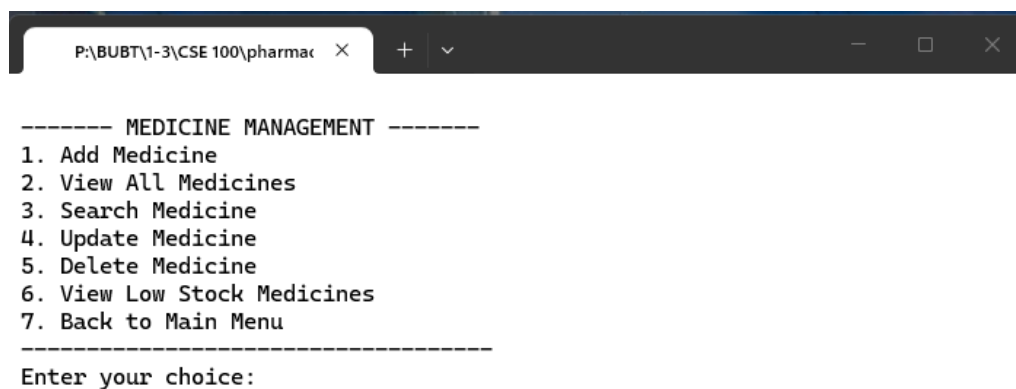


Figure 5.5: Add Medicine (Admin)

```
P:\BUBT\1-3\CSE 100\pharmac X + v - □ X

----- SALES MANAGEMENT -----
1. Sell Medicine
2. Back to Main Menu
-----

Enter your choice: 1

Find medicine by:
1. Medicine ID
2. Medicine Name
Enter your choice: 2
Enter Medicine Name (or part of it): NAPA
Medicine: NAPA 500mg | Available Stock: 100 | Unit Price: 1.20
Enter quantity to sell: 10

Sale successful. Transaction ID: TXN001
10 unit(s) of NAPA 500mg sold for 12.00 total. Date: Aug-20-2026

Press Enter to continue...
```

Figure 5.6: Sell Medicine

```
P:\BUBT\1-3\CSE 100\pharmac X + v - □ X

----- REPORTS -----
1. Inventory Report
2. Sales Report
3. Low Stock Report
4. Back to Main Menu
-----

Enter your choice: 1

===== INVENTORY REPORT =====

ID      Name          Generic Name      Category    Price    Qty
-----
279     NAPA 500mg       Paracetamol       Tablet      1.20     90
269     Finix 20mg       Rabeprazole Sodium Tablet      7.00     50
272     Ceevit 250mg     Vitamin C         Tablet      1.90     10
248     Remmo 20mg       Esomeprazole      Capsule     10.00     4

Total medicines: 4

Press Enter to continue...
```

Figure 5.7: Inventory Report

```

P:\BUBT\1-3\CSE 100\pharmac x + -
----- REPORTS -----
1. Inventory Report
2. Sales Report
3. Low Stock Report
4. Back to Main Menu
-----
Enter your choice: 2

=====
SALES REPORT
=====
TxnID  MedID  Medicine      Qty  Unit Price Total    Date      Seller
-----
TXN001  279    NAPA 500mg    10   1.20    12.00    Aug-20-2026  admin
TXN002  272    Ceevit 250mg   2    1.90     3.80    Aug-20-2026  staff
-----
Total Transactions: 2
Total Revenue: 15.80
=====

Press Enter to continue...

```

Figure 5.8: Sales Report

```

P:\BUBT\1-3\CSE 100\pharmac x + -
----- DASHBOARD -----
1. Medicine Management
2. Sales Management
3. Reports
4. Register New User
5. Logout
-----
Enter your choice: 4

--- Register New User ---
Username: staff
Password: staff123
Role (A = Admin, S = Staff): S
User 'staff' registered successfully as Staff.

Press Enter to continue...

```

Figure 5.9: Register New User

5.2.2 Panel B — Staff Interface

The Staff dashboard does not show the Register New User option. Add, Update, and Delete Medicine additionally require the admin password before proceeding.

```

P:\BUBT\1-3\CSE 100\pha x + -
----- DASHBOARD -----
1. Medicine Management
2. Sales Management
3. Reports
5. Logout
-----
Enter your choice:

```

Figure 5.10: Staff Dashboard

```
P:\BUBT\1-3\CSE 100\pharmaz X + - □ X

----- MEDICINE MANAGEMENT -----
1. Add Medicine
2. View All Medicines
3. Search Medicine
4. Update Medicine
5. Delete Medicine
6. View Low Stock Medicines
7. Back to Main Menu
-----

Enter your choice: 1

Staff must enter Admin password to add medicine.
Admin password: admin123

--- Add New Medicine ---
Medicine ID:
```

Figure 5.11: Add Medicine (Staff, Admin password prompt)

```
P:\BUBT\1-3\CSE 100\pharmac X + - □ X

----- SALES MANAGEMENT -----
1. Sell Medicine
2. Back to Main Menu
-----

Enter your choice: 1

Find medicine by:
1. Medicine ID
2. Medicine Name
Enter your choice: 2
Enter Medicine Name (or part of it): Ceevit
Medicine: Ceevit 250mg | Available Stock: 10 | Unit Price: 1.90
Enter quantity to sell: 2

Sale successful. Transaction ID: TXN002
2 unit(s) of Ceevit 250mg sold for 3.80 total. Date: Aug-20-2026
Warning: Ceevit 250mg is now low on stock (8 remaining).

Press Enter to continue...
```

Figure 5.12: Sell Medicine

5.2.3 Login Credentials

The following demo credentials were used for testing and demonstration:

Role	Username	Password
Admin	admin	admin123
Staff	staff	staff123

Note: these credentials only exist because they were manually created during testing (the admin account through the first-run setup, the Staff account through Register New User). A fresh copy of the program does not ship with any default account; the first person to run it is prompted to create the initial Admin account.

Chapter 6

CONCLUSION

6.1 Conclusion

This project set out to replace the manual, paper-based way that small pharmacies typically track medicine stock and sales with a simple, offline, console-based system. By the end of the semester, all five originally proposed modules, Login, Medicine Management, Inventory Management, Sales Management, and Reports, were implemented, tested, and integrated into a single working program. The system also went beyond the original proposal in two respects: it supports registering multiple Admin and Staff accounts rather than being limited to one fixed account of each, and medicine records include a generic name field in addition to the fields originally planned.

Working in an incremental, module-by-module way suited a three-person team well. It let us divide work along module boundaries, test each part on its own before integrating it, and still have a demonstrable system at every stage of the semester. The project was also selected to represent the class at the BUBT Software Development Idea Pitching & Project Showcasing event, which we treat as a reasonable indication that the system and its presentation were judged to be solid for an undergraduate project of this scope.

6.2 Limitation

- The system does not currently lock a user out after repeated failed login attempts, although this was part of the original design and remains something we intend to add.
- The application is single-user only. It does not support concurrent access from multiple machines or users at the same time.
- There is no graphical user interface. All interactions happen through the console/terminal.
- Expiry date tracking is not included in this version. Medicine expiry must be managed manually.
- Reports are displayed on the console screen only. There is no option to export or save a report to a file or print it.
- The system is primarily developed and tested on Windows. Limited testing may be performed on Linux. The system has not been tested on macOS, and compatibility with macOS is not guaranteed.

6.3 Future Works

- Lock the program after three failed login attempts, as originally planned.
- Expiry date tracking with automatic alerts when a medicine is near expiry.
- Supplier and vendor management to track from whom medicines were purchased.
- Report export to a .txt or .csv file so records can be saved permanently or shared.
- A graphical user interface, using a C GUI library or by migrating to another language.
- Barcode-based medicine search for faster lookup at the sales counter.

REFERENCES

- [1]. B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.
- [2]. MedEx Bangladesh, Medicine information database for the Bangladesh market. Retrieved from: <https://medex.com.bd>
- [3]. W. Royce, "Managing the Development of Large Software Systems," Proceedings of IEEE WESCON, vol. 26, pp. 1-9, 1970.
- [4]. R. S. Pressman, Software Engineering: A Practitioner's Approach, 8th ed. New York: McGraw-Hill, 2014.
- [5]. GNU Compiler Collection (GCC), Free Software Foundation. Retrieved from: gcc.gnu.org
- [6]. Microsoft, Visual Studio Code Documentation. Retrieved from: code.visualstudio.com/docs

APPENDIX A

Appendix A

The complete source code for this project has not been included in this report. It is hosted on GitHub and can be accessed using the link below.

Repository Link: <https://github.com/makhzumbinharun/pharmacy-inventory-system>

Project Title: Pharmacy Inventory Management System

Team Name: Three Knights

Student ID	Name	Role
20254103279	Makhzum-Bin-Harun	Team Leader & Core Programmer
20254103269	MD. Rafiul Islam	Core Programmer & SQA
20254103272	Estiak Ahmed Turnab	Core Programmer

Supervised by: Mastura Sadaf, Lecturer, Department of CSE, BUBT

A.1 Project Summary

The Pharmacy Inventory Management System is a console-based application built in C that helps small pharmacies handle their daily operations, such as managing medicine records, tracking stock, and recording sales. The system supports two types of users, Admin and Staff, with different levels of access depending on the action being performed.

A.2 Technology Stack

- Language: C (C11 standard)
- Compiler: GCC (MinGW-w64 on Windows, GCC via apt on Linux)
- Editor: Visual Studio Code
- Data Storage: Binary .dat files, no external database used
- Version Control: Git and GitHub

A.3 Repository Structure

```
pharmacy-inventory-system/  
├── src/  
│   ├── main.c  
│   ├── auth.c  
│   ├── medicine.c  
│   ├── inventory.c  
│   ├── sales.c  
│   └── reports.c  
│   └── utils.c  
├── include/  
│   ├── auth.h  
│   ├── medicine.h  
│   ├── inventory.h  
│   ├── sales.h  
│   └── reports.h  
│   └── utils.h  
├── data/ # Created automatically at runtime (not committed)  
│   ├── users.dat  
│   ├── medicines.dat  
│   └── sales.dat  
├── docs/  
│   ├── proposal.pdf  
│   └── screenshots/  
├── .gitignore  
└── README.md
```

A.4 How to Build and Run

The project can be compiled and run using GCC on both Windows and Linux.

Windows (MinGW/GCC):

```
gcc src/*.c -o pharmacy -Iinclude  
pharmacy.exe
```

Linux:

```
gcc src/*.c -o pharmacy -Iinclude  
./pharmacy
```

A.5 License

The repository is released under the MIT License, which permits reuse and modification of the code with proper attribution.